



New Prince Shri Bhavani

College of Engineering and Technology



Tambaram-Velachery Main Road, Santhosapuram, Chennai-600 073

DEPARTMENT OF CYBER SECURITY

LAB MANUAL

U23CS402 –DESIGN AND ANALYSIS OF ALGORITHM (REGULATION - 2023)

II YEAR/IV SEMESTER

ISBN 978-933411344-0



9

789334

113440



NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



VISION OF THE INSTITUTION:

To be a globally recognized academic institution and thereby contribute to technological and socio-economic development of the society.

MISSION OF THE INSTITUTION:

1. To develop the needed resources and infrastructure, and to establish a conducive ambience for the teaching- learning process.
2. To Engage Committed members of faculty who will infuse subject knowledge with latest teaching pedagogies
3. To ignite the desire for higher learning, research and entrepreneurship and equip them to face the global challenges
4. To engage the alumni professionals as productive partners with the current students to help enrich and enhance their student life.

VISION OF THE DEPARTMENT

Foster and nurture students, inspiring them to be Cyber Security entrepreneurs, by offering a viable and research-centric educational platform for building a robust, resilient, and attack-free digital society.

MISSION OF THE DEPARTMENT

- M1:** Abet students to be Cyber Security Entrepreneurs with a hands-on, research-based learning for a secure digital society.
- M2:** Strengthen collaborative project learning to address global challenges whilst preserving human and ethical values.
- M3:** Encourage students to help society to restore human safety and security in digital world.



NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



PROGRAM EDUCATIONAL OBJECTIVES:

PEO 1: Skilled Cyber Security entrepreneurs, use their knowledge for building a safe and secure digital world.

PEO 2: Apply skills to develop innovative solutions for evolving digital challenges in Cyber Security.

PEO 3: Uphold ethical standards, contributing responsibly to Cyber Security and staying updated with the latest technologies.

PROGRAM SPECIFIC OUTCOME (PSO):

PSO 1: Expertise to design varying complexity with help of cryptography, web development, deep learning by adhering software development life cycle.

PSO 2: Proficiency to develop data, resource, and asset protection strategies for organizations, processes, people, and individuals through Cyber Security-centric skills.

PROGRAM OUTCOMES(POS):

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.



**NEW PRINCE SHRI BHAVANI
COLLEGE OF ENGINEERING AND TECHNOLOGY**

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



U23CS402

DESIGN AND ANALYSIS OF ALGORITHMS

L	T	P	C
3	0	2	4

COURSE OBJECTIVES:

- To understand and apply algorithm analysis techniques.
- To understand efficiency and design techniques of different algorithms.
- To understand the basic concepts of NP completeness and approximation algorithms.

LIST OF EXPERIMENTS :

1. Implement Linear Search. Determine the time required to search for an element.
2. Repeat the experiment for different values of n , the number of elements in the list. Compare and plot the time taken.
3. Implement Binary Search. Determine the time required to search for an element.
4. Repeat the experiment for different values of n , the number of elements in the list. Compare and plot the time taken.
5. Given a text $[0...n-1]$ and a pattern $pat[0...m-1]$, write a function `search(char pat[], char txt[])`.
6. Develop a program to implement graph traversal using Breadth First Search.
7. Develop a program to implement graph traversal using Depth First Search.

TEXT BOOKS:

1. S. Sridhar, "Design and Analysis of Algorithms", 2nd Edition, Oxford University Press, 2023.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, "Introduction to Algorithms", 4th Edition, MIT Press, 2022.

REFERENCES:

1. Anany Levitin, "Introduction to the Design and Analysis of Algorithms", 3rd Edition, Pearson Education, 2018.
2. Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, "Fundamentals of Computer Algorithms", 2nd Edition, Universities Press, 2018.
3. Ethem Alpaydin, "Introduction to Algorithms", 4th Edition, MIT Press, 2020.

TOTAL: 30 PERIODS



NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



COURSE OUTCOME – CO - PO

MAPPING COURSE OUTCOME

Subject Code :		U23CS402	Semester :	IV
Subject Name :		DESIGN AND ANALYSIS OF ALGORITHMS		
Course Outcome				
CO1	Analyze search, sort, and string matching algorithms to compute their efficiency.			
CO2	Apply graph algorithms to solve problems and analyze their efficiency.			
CO3	Apply algorithm design techniques like divide & conquer, dynamic programming, and greedy techniques to solve problems.			
CO4	Apply the state space tree method for solving problems.			
CO5	Solve problems using approximation algorithms and NP-Complete concepts.			

CO-PO-PSO MAPPING:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	2	2	1	-	-	1	1	2	-	-	2	2
CO2	3	2	1	2	1	-	-	1	1	2	-	-	2	2
CO3	3	2	1	2	1	-	-	1	1	2	-	-	2	2
CO4	3	2	1	2	1	-	-	1	1	2	-	-	2	2
CO5	3	2	1	2	1	-	-	1	1	2	-	-	2	2
Total	15	11	6	10	5	-	-	5	5	10	-	-	10	10
No. Of Cos mapping with POs	5	5	5	5	5	-	-	5	5	5	-	-	5	5
Average	3	2.2	1.2	2	1	-	-	1	1	2	-	-	2	2
Round (Average)	3	2	1	2	1	-	-	1	1	2	-	-	2	2



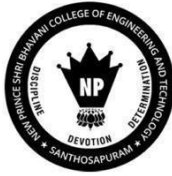
NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



INDEX

S.NO	NAME OF THE EXPERIMENT	CO's MAPPE D	PO's & PSO's MAPPED	PG. NO.
1,2	Implement linear search	CO1	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	8-9
3,4	Implement recursive binary search	CO1	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	10-11
5	String matching algorithm	CO2	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	12-13
6	Implement Breadth First Search	CO2	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	14-15
7	Implement Depth First Search	CO2	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	16-17
	CONTENT BEYOND SYLLABUS			
8	Insertion sort and Heap sort	CO3	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	
9	Implement N Queens problem	CO4	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	
10	Implement Traveling Salesman problem using Approximation algorithm	CO5	PO1,PO2,PO3, PO4,PO5,PO8,P O9,PO10,PSO1, PSO2	



NEW PRINCE SHRI BHAVANI COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to Anna University | Approved by AICTE | Accredited by NAAC
Tambaram - Velachery Main Road, Santhosapuram, Chennai-600 073.



Ex.No: 1,2

IMPLEMENT LINEAR SEARCH

Date:

AIM:

To implement Linear search and determine the time required to search an element and plot a graph of time Taken versus n.

ALGORITHM:

Step 1: Start.

Step 2: Define linear search function.

Step 3: Define run-experiment to find the time taken.

Step 4: Create an array with number of elements in the list to find the time taken. Step 5: Plot the time taken vs n using pyplot.

Step 6: Stop.

PROGRAM:

```
import time
import matplotlib.pyplot as plt
def linear_search(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1

def run_experiment(n):
    arr = list(range(n))
    x = n - 1
    start = time.time()
    result = linear_search(arr, n)
    end = time.time()
    print(n, ":", end - start, "is the time taken")
    return end - start

ns = [10, 100, 1000, 10000, 100000, 1000000]
```



```

times = [run_experiment(n) for n in ns]
# Plot the results
plt.plot(ns, times)
plt.xlabel("n")
plt.ylabel("Time (seconds)")
plt.title("Time taken for linear search")
plt.show()

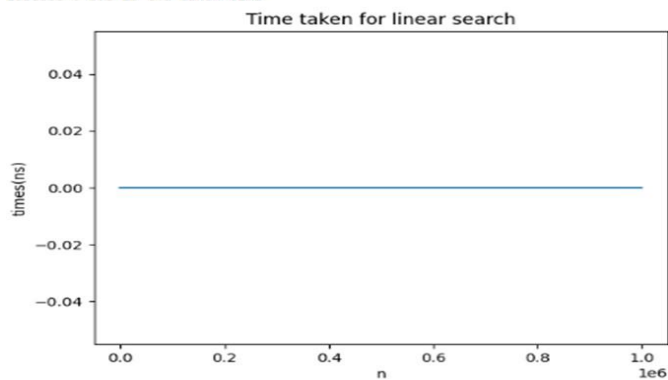
```

OUTPUT:

```

10 : 0.0 is the taken time
100 : 0.0 is the taken time
1000 : 0.0 is the taken time
10000 : 0.0 is the taken time
100000 : 0.0 is the taken time
1000000 : 0.0 is the taken time

```



RESULT:

Thus the program to implement Linear search and determine the time required to search an element and plot a graph of time Taken versus n has been executed successfully

Ex.No:3,4 IMPLEMENT RECURSIVE BINARY SEARCH**Date:****AIM:**

To implement recursive binary search and determine the time required to search an element and plot a graph of time Taken versus n.

ALGORITHM:

Step1: Start.

Step 2: Define recursive binary search.

Step 3: Import time module to determine the time taken to search an element. Step 4: Plot the time taken vs n using pyplot.

Step 5: Stop.

PROGRAM:

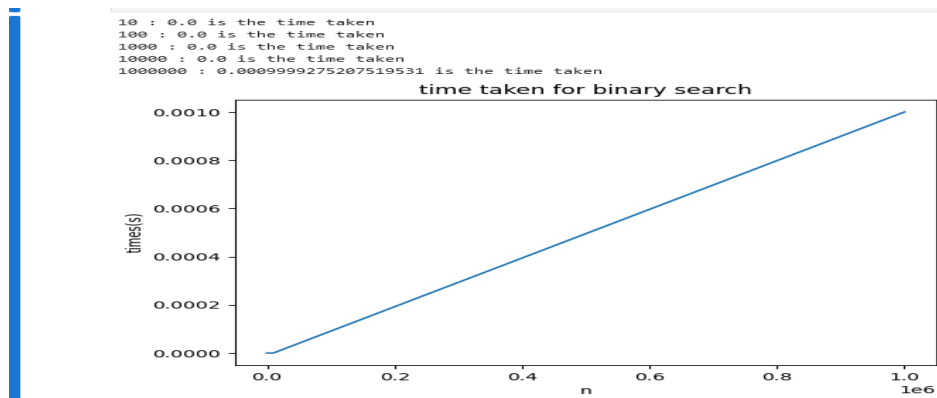
```
import time
import matplotlib.pyplot as plt
def binary_search(arr, low, high, x):
    if low <= high:
        mid = (low + high) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            return binary_search(arr, mid + 1, high, x)
        else:
            return binary_search(arr, low, mid - 1, x)
    else:
        return -1
def run_experiment(n):
    arr = list(range(n))
    x = n - 1
    start_time = time.time()
    result = binary_search(arr, 0, n - 1, x)
    end_time = time.time()
```

```

    print(n, ":", end_time - start_time, "seconds")
    return end_time - start_time
ns = [10, 100, 1000, 10000, 100000]
times = [run_experiment(n) for n in ns]
plt.plot(ns, times)
plt.xlabel("n (Array size)")
plt.ylabel("Time (seconds)")
plt.title("Time taken for Binary Search")
plt.show()

```

OUTPUT:



RESULT:

Thus the implementation of binary search has been executed successfully.

Ex.No:5

Date:

Given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that $n > m$.

AIM:

To write a program for a text txt[0...n-1] and a pattern pat[0...m-1], using a function search(char pat[], char txt[]) that prints all occurrences of pat[] in txt[].

ALGORITHM:

- Step 1: Start.
- Step 2: Get the text string and pattern string.
- Step 3: Perform search operations.
- Step 4: Print the pattern found.
- Step 5: Stop.

PROGRAM:

```
def search(pat, txt):  
    M = len(pat)  
    N = len(txt)  
    for i in range(N - M + 1):  
        j = 0  
        while j < M:  
            if txt[i + j] != pat[j]:  
                break  
            j += 1  
  
        if j == M:  
            print("Pattern found at index", i)  
if __name__ == "__main__":  
    txt = "AABAACAADAABAAABAA"  
    pat = "AABA"  
    search(pat, txt)
```

OUTPUT:

Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

RESULT:

The program to implement txt and pat using a function `search(char pat[],char txt[])` is executed successfully.

Ex. No: 6

GRAPH TRAVERSAL - BREADTH FIRST SEARCH

Date:

AIM:

To implement graph traversal using Breadth First Search.

ALGORITHM:

Step 1: Start.

Step 2: Put any one of the graph's vertices at the back of the queue.

Step 3: Take the front item of the queue and add it to the visited list.

Step 4: Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.

Step 5: Keep continuing steps two and three till the queue is empty.

Step 6: Stop.

PROGRAM:

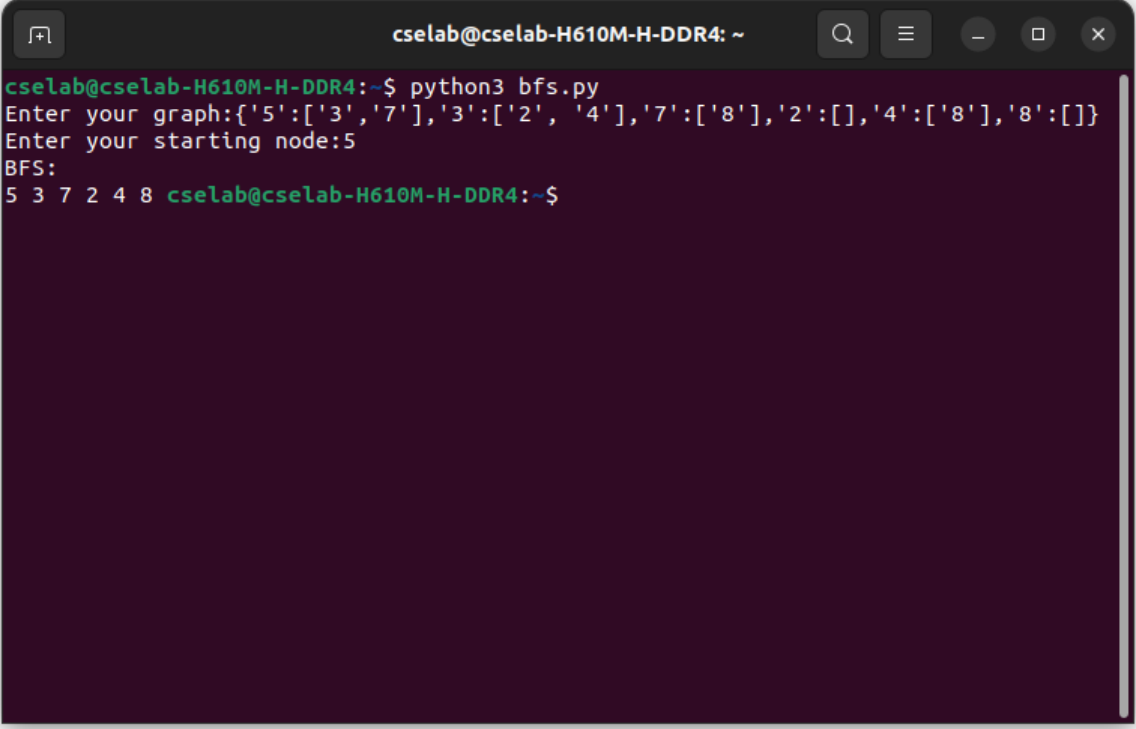
```
graph = eval(input("Enter your graph "))
node = input("Enter your starting node: ")
visited = []
queue = []
```

```
def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        m = queue.pop(0)
        print(m, end=" ")
        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
```

```
        queue.append(neighbour)
print("BFS:")
bfs(visited, graph, node)
```

OUTPUT:

A terminal window with a dark background and light green text. The window title is 'cselab@cselab-H610M-H-DDR4: ~'. The prompt is 'cselab@cselab-H610M-H-DDR4:~\$'. The user enters 'python3 bfs.py'. The program prompts 'Enter your graph:' and the user enters '{'5':['3','7'],'3':['2','4'],'7':['8'],'2':[],'4':['8'],'8':[]}'. The program prompts 'Enter your starting node:' and the user enters '5'. The program outputs 'BFS:' followed by the sequence '5 3 7 2 4 8' on the same line. The prompt 'cselab@cselab-H610M-H-DDR4:~\$' is visible at the end of the line.

```
cselab@cselab-H610M-H-DDR4:~$ python3 bfs.py
Enter your graph:{'5':['3','7'],'3':['2','4'],'7':['8'],'2':[],'4':['8'],'8':[]}
Enter your starting node:5
BFS:
5 3 7 2 4 8 cselab@cselab-H610M-H-DDR4:~$
```

RESULT:

Thus, graph traversal using Breadth First Search has been implemented successfully.

Ex. No: 7

GRAPH TRAVERSAL - DEPTH FIRST SEARCH

Date:

AIM:

To implement graph traversal using Depth First Search.

ALGORITHM:

Step 1: Start.

Step 2: Put any one of the graph's vertex on top of the stack.

Step 3: Take the top item of the stack and add it to the visited list of the vertex.

Step 4: Create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.

Step 5: Keep repeating steps 2 and 3 until the stack is empty.

Step 6: Stop.

PROGRAM:

```
graph = eval(input("Enter your graph "))
node = input("Enter your starting node: ")
visited = set()
```

```
def dfs(visited, graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

print("DFS:")
dfs(visited, graph, node)
```

OUTPUT

```
cselab@cselab-H610M-H-DDR4: ~  
cselab@cselab-H610M-H-DDR4:~$ python3 dfs.py  
Enter your graph: {'5': ['3', '7'], '3': ['2', '4'], '7': ['8'], '2': [], '4': ['8'], '8': []}  
Enter your starting node: 5  
DFS:  
5 3 2 4 8 7 cselab@cselab-H610M-H-DDR4:~$
```

RESULT:

Thus, graph traversal using Depth First Search has been implemented successfully.

Ex.No:8

INSERTION SORT & HEAP SORT

Date:

AIM:

To write a program to sort a given set of elements using insertion sort and heap sort and to determine time required for sorting plot a graph of time taken vs n.

ALGORITHM:

Step 1: Start.

Step 2: Create the array for sorting.

Step 3: Perform insertion sort and heap sort.

Step 4: Display the time required for sorting.

Step 5: Plot the graph for time taken vs n.

Step 6: End.

PROGRAM:

```
import time
import matplotlib.pyplot as plt
import random

def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

def heap_sort(arr):
    n = len(arr)

    def heapify(arr, n, i):
        largest = i
        l = 2 * i + 1
        r = 2 * i + 2
        if l < n and arr[l] > arr[largest]:
```

```

        largest = l
    if r < n and arr[r] > arr[largest]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

for i in range(n//2 - 1, -1, -1):
    heapify(arr, n, i)

for i in range(n-1, 0, -1):
    arr[0], arr[i] = arr[i], arr[0]
    heapify(arr, i, 0)

def measure_sort_time(sort_func, arr):
    start_time = time.time()
    sort_func(arr)
    end_time = time.time()
    return end_time - start_time

def create_random_list(n):
    return [random.randint(1, 1000) for _ in range(n)]

n_values = [10, 100, 1000, 10000]
insertion_times = []
heap_times = []

for n in n_values:
    arr = create_random_list(n)

    # Measure time for Insertion Sort
    insertion_time = measure_sort_time(insertion_sort, arr)

    # Measure time for Heap Sort
    heap_time = measure_sort_time(heap_sort, arr)

```

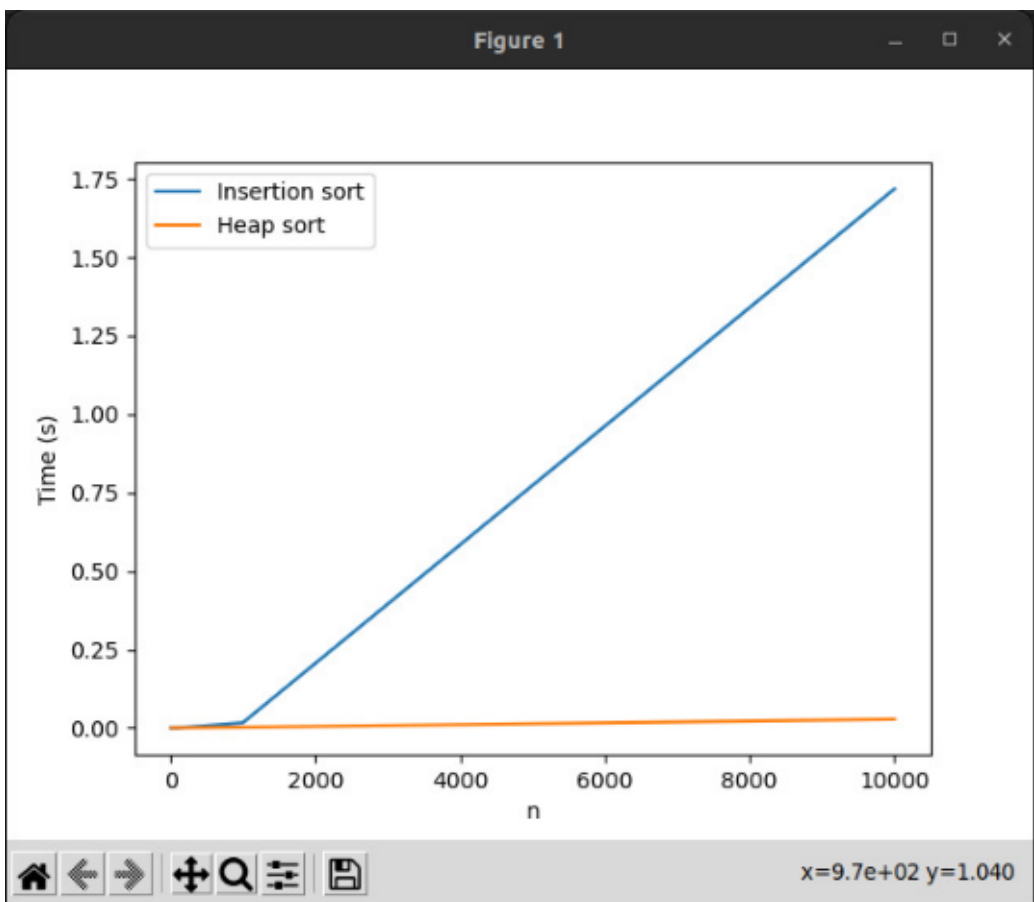
```
insertion_times.append(insertion_time)
heap_times.append(heap_time)

print(f'n = {n}: Insertion sort took {insertion_time:.6f} seconds, Heap sort took
{heap_time:.6f} seconds')

# Plot the results
plt.plot(n_values, insertion_times, label="Insertion sort")
plt.plot(n_values, heap_times, label="Heap sort")
plt.xlabel("n (Array Size)")
plt.ylabel("Time (s)")
plt.legend()
plt.title("Comparison of Insertion Sort and Heap Sort")
plt.show()
```

OUTPUT:

```
Terminal
user@user-H610M-H-DDR4:~$ python3 sort.py
n = 10: Insertion sort took 3.0994415283203125e-06 seconds, Heap sort took 1.049
041748046875e-05 seconds
n = 100: Insertion sort took 0.0001571178436279297 seconds, Heap sort took 0.000
13065338134765625 seconds
n = 1000: Insertion sort took 0.016654491424560547 seconds, Heap sort took 0.002
0189285278320312 seconds
n = 10000: Insertion sort took 1.7182378768920898 seconds, Heap sort took 0.0279
3264389038086 seconds
```



RESULT:

Thus, the program has been executed successfully.

Ex.No:9

N QUEEN PROBLEM USING BACKTRACKING

Date:

AIM:

To write a program to implement N Queen problem using Backtracking.

ALGORITHM:

Step 1: Start.

Step 2: Initialize an empty chessboard of size $N \times N$.

Step 3: Start with the leftmost column and place a queen in the first row of that column.

Step 4: Move to the next column and place a queen in the first row of that column.

Step 5: Repeat step 3 until either all N queens have been placed or it is impossible to place a queen in the current column without violating the rules of the problem.

Step 6: If all N queens have been placed, print the solution.

Step 7: If it is not possible to place a queen in the current column without violating the rules of the problem, backtrack to the previous column

Step 8: Remove the queen from the previous column and move it down one row.

Step 9: Repeat steps 4-7 until all possible configurations have been tried.

Step 10: stop.

PROGRAM:

```
import numpy as np
from itertools import permutations
def queen(a):
    for cl in permutations(range(a)):
        mtrx = np.zeros(a*a).reshape(a,a)
        check = 0
        for i in range(a):
            if check != i*(i-1)/2:
                break
            for j in range(1,a):
                if i >= j:
                    if cl[i] == cl[i-j]+j or cl[i] == cl[i-j]-j:
                        check = 0
                        break
                else:
                    check += 1
            else:
                continue
```

```

if check == a*(a-1)/2:
for r in range(a):
mtrx[r][cl[r]] = 1
yield mtrx
else:
pass
num = int(input())
print(f"Input: {num}\a") print("solution:\a")
for m in queens(num):
print(m,"\a")

```

OUTPUT:

```

user@user-H510M-H: ~
user@user-H510M-H:~$ python3 n_queen.py
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
user@user-H510M-H:~$

```

RESULT:

Thus, the implementation of N Queen problem using Backtracking has been executed successfully.

Ex.No:10

Date:

Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation

AIM:

To Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation.

ALGORITHM:

Step 1 : Start

Step 2 : import required modules in the program

Step 3: Find the optimal solution for Travelling Salesperson problem

Step 4: Solve the same problem using approximation algorithm

Step 5: Determine the error in the approximation

Step 6 : Stop

PROGRAM:

```
import sys
def tsp_optimal_solution(distances):
    n = len(distances)
    visited = set()
    optimal_cost = 0
    optimal_path = [0]
    visited.add(0)
    while len(visited) < n:
        min_distance = sys.maxsize
        min_index = -1
        for i in range(n):
            if i not in visited and distances[optimal_path[-1]][i] < min_distance:
                min_distance = distances[optimal_path[-1]][i]
                min_index = i
        visited.add(min_index)
        optimal_path.append(min_index)
        optimal_cost += min_distance
        optimal_path.append(0)
        optimal_cost += distances[optimal_path[-2]][0]
    return optimal_cost, optimal_path
def tsp_approx_solution(distances):
    n = len(distances)
    visited = set()
    approx_cost = 0
    approx_path = [0]
    visited.add(0)
    while len(visited) < n:
```

```

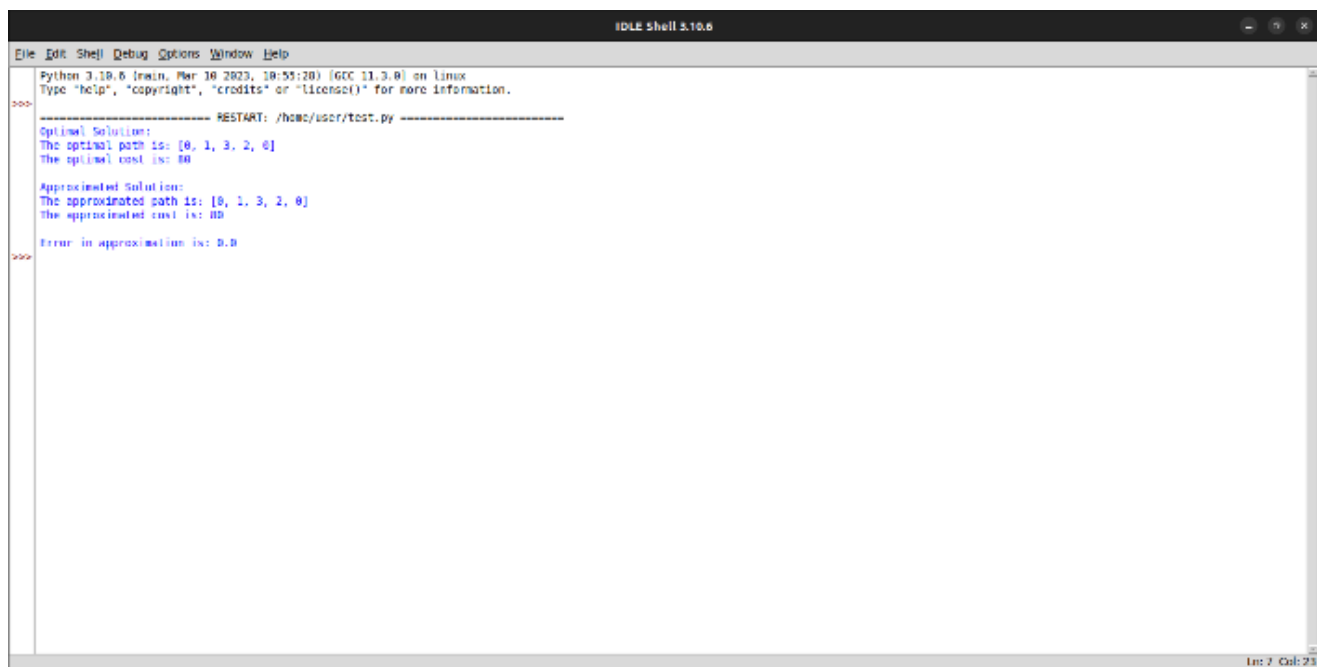
min_distance = sys.maxsize
min_index = -1
for i in range(n):
    if i not in visited and distances[approx_path[-1]][i] < min_distance:
        min_distance = distances[approx_path[-1]][i]
        min_index = i
    visited.add(min_index)
    approx_path.append(min_index)
    approx_cost += min_distance
    approx_path.append(0)
    approx_cost += distances[approx_path[-2]][0]
return approx_cost, approx_path

def calculate_error(approx_cost, optimal_cost):
    return (approx_cost - optimal_cost) / optimal_cost

if __name__ == '__main__':
    distances = [[0, 10, 15, 20],
                 [10, 0, 35, 25],
                 [15, 35, 0, 30],
                 [20, 25, 30, 0]]
    optimal_cost, optimal_path = tsp_optimal_solution(distances)
    print("Optimal Solution:")
    print("The optimal path is:", optimal_path)
    print("The optimal cost is:", optimal_cost)
    approx_cost, approx_path = tsp_approx_solution(distances)
    print("\nApproximated Solution:")
    print("The approximated path is:", approx_path)
    print("The approximated cost is:", approx_cost)
    error = calculate_error(approx_cost, optimal_cost)
    print("\nError in approximation is:", error)

```

OUTPUT:



```
Python 3.10.6 (main, Mar 10 2023, 18:55:20) (GCC 11.3.0) on linux
Type "help", "copyright", "credits" or "license()" for more information.

>>> ----- RESTART: /home/user/test.py -----
Optimal Solution:
The optimal path is: [0, 1, 3, 2, 0]
The optimal cost is: 0.0

Approximated Solution:
The approximated path is: [0, 1, 3, 1, 0]
The approximated cost is: 0.0

Error in approximation is: 0.0

>>>
```

RESULT :

Thus the python program for travelling salesperson problem has been executed successfully.

